

# Flujo local y sin nube para actualizar un archivo .mmb desde transacciones bancarias

## Resumen ejecutivo

La conclusión central es esta: si tu archivo `.mmb` corresponde a **Money Manager Ex** (MMEX), no estás ante un formato cerrado ni “mágico”, sino ante una base de datos SQLite; la documentación oficial de MMEX incluso dice explícitamente que el `.mmb` no es propietario y que MMEX usa SQLite. Además, MMEX distingue entre `.mmb` y una variante cifrada `.emb`. Eso cambia por completo la estrategia: sí es posible automatizar un flujo 100% local para leer correos, adjuntos, exportes bancarios y entradas manuales, normalizar todo en una base canónica local y luego **importar a MMEX por CSV/QIF** o, en una segunda fase más avanzada, **escribir directamente en el `.mmb`**. <sup>1</sup>

Mi recomendación objetiva es **no** conectar “correo → `.mmb`” de forma directa desde el primer día. Recomiendo una arquitectura en dos capas: **(1) bandeja de ingestión + base canónica SQLite local**, y **(2) exportador/importador hacia MMEX**. Las razones son cuatro. Primero, tú mismo ya pones la restricción decisiva: no todas las transacciones llegan por correo, así que el correo sirve como señal temprana, pero no como libro mayor completo. Segundo, OFX, QIF, CSV y MT940 tienen semánticas distintas, y conviene reconciliarlas antes de tocar el archivo final. Tercero, MMEX ya trabaja bien con CSV/QIF y maneja estados como *Followup* y *Duplicate*, lo que encaja muy bien con una fase humana de revisión. Cuarto, separar la capa canónica de la capa `.mmb` te da idempotencia, deduplicación, auditoría y rollback. <sup>2</sup>

En términos prácticos, el stack más sólido para un flujo local y offline es: **Thunderbird o un sincronizador local de correo**, **Python 3 + SQLite** como motor de ingestión, **pypdf/PyMuPDF/pdfplumber** para PDF nativos, **Tesseract + OCRmyPDF/NAPS2** para OCR, **RapidFuzz** para deduplicación difusa, y **MMEX** como destino final. Si la prioridad es robustez, la ruta inicial más segura es **exportar a CSV compatible con MMEX** e importar por cuenta; si la prioridad es automatización total, la ruta avanzada es **insertar en las tablas de MMEX con transacciones SQLite, validación de esquema y copias de respaldo previas**. <sup>3</sup>

## Qué es realmente un archivo .mmb y cómo abordarlo

Asumiendo que tu `.mmb` es de MMEX —la hipótesis más razonable en finanzas personales—, la base técnica está bastante clara. El manual oficial de MMEX dice que el fichero `.mmb` **no es propietario** y que MMEX usa **SQLite**. La propia documentación de MMEX para Android vuelve a describir el `.mmb` como el archivo de base de datos principal, y la documentación de seguridad indica que, si quieres cifrado, MMEX trabaja con una variante `.emb`. Por tanto, para un flujo local, `.mmb` es perfectamente inspeccionable con herramientas SQLite estándar; `.emb` requiere un tratamiento aparte y no debería tocarse con `sqlite3` “a ciegas”. <sup>4</sup>

La segunda pieza es el esquema. El repositorio del proyecto publica la base/snapshot del esquema con tablas como `ACCOUNTLIST_V1`, `CHECKINGACCOUNT_V1`, `PAYEE_V1`, `CATEGORY_V1`, `SUBCATEGORY_V1`, `TAG_V1`, `TAGLINK_V1`, `ATTACHMENT_V1`, `CUSTOMFIELD_V1` y `CUSTOMFIELDDATA_V1`. Eso significa que MMEX no solo guarda movimientos: también tiene una capa

nativa para cuentas, beneficiarios/payas, etiquetas, adjuntos y campos personalizados. Esta última parte es importante, porque tus campos “extra” —por ejemplo `source_user`, `card_last4`, `raw_text`, `source_ref` y `source_type`— no tienen por qué acabar aplastados en una sola nota; puedes persistirlos como etiquetas, adjuntos o custom fields. <sup>5</sup>

La consecuencia ingenieril es importante: **es posible** generar o actualizar `.mmb` de forma local, pero **no conviene** empezar escribiendo directamente sobre la base final. La práctica correcta es: abrir en solo lectura, verificar tablas con `PRAGMA table_info(...)`, crear una copia de trabajo, probar inserciones sobre una copia, y solo entonces tocar la base operativa. MMEX, además, insiste en respaldar regularmente la base y mantener copias en dispositivos separados. <sup>6</sup>

Si tu `.mmb` no fuera de MMEX, la estrategia de contingencia sería un pequeño proceso de ingeniería inversa local: comprobar firma/magic (`SQLite format 3`, ZIP, etc.), extraer cadenas legibles, comparar bytes antes y después de una transacción manual, observar accesos del programa al archivo con Process Monitor de Microsoft <sup>7</sup>, y, si el formato fuera binario opaco, describirlo con Kaitai Struct o buscar contenedores incrustados con Binwalk. Para SQLite o SQLCipher, DB Browser for SQLite y SQLiteStudio son los instrumentos obvios de inspección. <sup>8</sup>

| Caso                          | Señal técnica                          | Herramientas locales                                            | Uso recomendable                                                |
|-------------------------------|----------------------------------------|-----------------------------------------------------------------|-----------------------------------------------------------------|
| <code>.mmb</code> de MMEX     | Manual oficial: SQLite, no propietario | <code>sqlite3</code> , DB Browser, SQLiteStudio                 | Inspección y actualización controlada. <sup>9</sup>             |
| <code>.emb</code> de MMEX     | Variante cifrada de la base            | MMEX, copia/transformación controlada, cifrado externo          | Mejor procesar en staging y convertir al final. <sup>10</sup>   |
| <code>.mmb</code> desconocido | Formato no documentado                 | Kaitai Struct, Binwalk, Procmon, <code>strings</code> , hexdump | Ingeniería inversa comparativa antes de escribir. <sup>11</sup> |

## Fuentes de transacciones y herramientas offline

Para tu caso conviene pensar en **dos familias de fuente: notificación y extracto**. La notificación es rápida y granular —correo, imagen, SMS exportado, PDF individual—, pero puede faltar o describir solo una autorización. El extracto —CSV, OFX, QIF, MT940— es más lento, pero está diseñado precisamente para intercambio o conciliación de información financiera. Mi recomendación es tratar el correo como fuente primaria de captura temprana y los exportes bancarios como fuente periódica de verdad contable. Esa recomendación se apoya en la propia naturaleza de OFX como estándar de intercambio financiero abierto, en MT940 como extracto estructurado de cuenta y en QIF/CSV como formatos de importación/exportación ampliamente usados por software financiero. <sup>12</sup>

| Fuente / formato                          | Qué te da                                                        | Parsers o herramientas offline                                                         | Observaciones                                                                                                                                                                                                                                   |
|-------------------------------------------|------------------------------------------------------------------|----------------------------------------------------------------------------------------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| CSV / TSV bancario                        | Tablas simples, pero dialecto variable                           | <code>csv</code> de Python, <code>pandas.read_csv</code> , <code>csvkit</code>         | CSV tiene una semántica mínima común, pero cada banco cambia nombres, separadores, codificaciones y signos; por eso conviene usar un adaptador por banco. <sup>13</sup>                                                                         |
| OFX                                       | Estándar abierto para banca, tarjetas, bill pay e inter-transfer | <code>ofxtools</code> , <code>ofxparse</code> , <code>libofx</code>                    | El grupo OFX de Financial Data Exchange <sup>14</sup> publica OFX Banking 2.3 y documenta que OFX 1.x fue SGML y OFX 2.x es XML; muy buen formato para importación estructurada. <sup>15</sup>                                                  |
| QIF                                       | Texto plano ASCII con cabecera por tipo de cuenta                | <code>quiffen</code> , <code>qifparse</code>                                           | QIF es abierto y editable a mano; útil como formato puente y como conversión si prefieres generar QIF desde tu base canónica. <sup>16</sup>                                                                                                     |
| MT940                                     | Extracto electrónico estructurado                                | <code>mt940</code>                                                                     | MT940 define bloques y campos como <code>20</code> , <code>25</code> , <code>28C</code> , <code>60F</code> , <code>61</code> y <code>86</code> ; es excelente para conciliación desde banca corporativa o portales empresariales. <sup>17</sup> |
| Correo <code>.eml</code> / mbox / Maildir | Cabeceras, cuerpo, MIME, adjuntos                                | <code>email</code> , <code>mailbox</code> , <code>imaplib</code> , <code>poplib</code> | La librería estándar de Python tiene buena base, y además existe documentación oficial en español para estas piezas. <sup>18</sup>                                                                                                              |
| Correo HTML / texto plano                 | Notificaciones y confirmaciones legibles                         | <code>BeautifulSoup</code> , <code>lxml</code> , <code>regex</code>                    | Muy útil para plantillas bancarias repetitivas; <code>BeautifulSoup</code> y <code>lxml</code> son suficientes para extracción offline. <sup>19</sup>                                                                                           |
| Outlook <code>.msg</code>                 | Correos exportados desde Outlook                                 | <code>extract-msg</code>                                                               | Buena salida si alguno de ustedes usa Outlook y archiva correos en <code>.msg</code> . <sup>20</sup>                                                                                                                                            |
| PDF con texto nativo                      | Comprobantes y estados exportados                                | <code>pypdf</code> , <code>pdfplumber</code> , <code>PyMuPDF</code>                    | Conviene intentar <b>primero</b> extracción nativa antes de aplicar OCR; es más preciso cuando el PDF ya contiene texto. <sup>21</sup>                                                                                                          |

| Fuente / formato       | Qué te da                                            | Parsers o herramientas offline                                           | Observaciones                                                                                                                                                                                |
|------------------------|------------------------------------------------------|--------------------------------------------------------------------------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| PDF escaneado / imagen | Boletas, capturas, foto de comprobante               | Tesseract ,<br>OCRmyPDF ,<br>pytesseract ,<br>Pillow , OpenCV ,<br>NAPS2 | OCR local total; Tesseract usa paquetes de idioma locales, OCRmyPDF añade capa OCR al PDF, NAPS2 ofrece flujo GUI multiplataforma para escaneo/OCR. <sup>22</sup>                            |
| Entrada manual         | Movimientos que nunca llegaron por correo ni exporte | formulario local, CSV local, la propia UI de MMEX                        | Para tu caso es imprescindible tener una cola manual, porque el correo no cubrirá todo. MMEX importa CSV/QIF/HTML/XML, y su flujo de transacciones soporta revisión posterior. <sup>23</sup> |

Donde sí hay documentación oficial en español, merece la pena aprovecharla: Python publica en español la documentación de `mailbox`, `email`, `imaplib`, `poplib`, `csv` y `sqlite3`; eso cubre buena parte del pipeline básico sin depender de tutoriales de terceros. <sup>24</sup>

## Arquitectura del pipeline offline

La arquitectura que mejor equilibra seguridad, auditabilidad y mantenimiento es una **plataforma local por carpetas + SQLite**. En términos simples: un buzón local sincronizado a disco, una carpeta de “drop” para exportes bancarios y capturas, una base canónica local donde entra todo, un motor de normalización y deduplicación, y al final un exportador hacia MMEX. Si quieres una sola frase-resumen: **haz que el `.mmmb` sea el destino, no el punto de integración**. Eso reduce muchísimo el riesgo de corrupción y te permite reimportar, rehacer o revertir sin perder trazabilidad. <sup>25</sup>

flowchart TD

```

A[Correo local IMAP/POP3 o .eml/.mbox] --> B[Extracción MIME y adjuntos]
C[Exportes bancarios CSV/OFX/QIF/MT940] --> D[Parsers por formato]
E[PDF nativo / PDF escaneado / imágenes] --> F[Texto nativo u OCR local]
G[Entrada manual] --> H[Formulario/CSV local]

```

```

B --> I[Normalización a esquema canónico]
D --> I
F --> I
H --> I

```

```

I --> J[SQLite staging local]
J --> K[Deduplicación + merge Yo/Esposa/Joint]
K --> L[Reglas de conciliación]
L --> M[Exportador CSV/QIF MMEX]
L --> N[Writer directo a .mmmb]
M --> O[Revisión en MMEX]
N --> O

```

J --> P[Adjuntos y texto original archivados]  
 0 --> Q[Backups cifrados locales]

## Herramientas recomendadas por etapa

| Etapas                  | Opción recomendada                        | Alternativas                             | Nota práctica                                                                                                                                                                        |
|-------------------------|-------------------------------------------|------------------------------------------|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Buzón local GUI         | Thunderbird de Mozilla <sup>26</sup>      | cliente similar con almacenamiento local | Thunderbird sincroniza IMAP para actividad online/offline y permite mantener mensajes descargados localmente; sirve muy bien como caché de trabajo. <sup>27</sup>                    |
| Sincronización headless | <code>getmail6</code>                     | <code>mbsync</code> / <code>isync</code> | <code>getmail6</code> soporta POP3/IMAP y entrega a Maildir/mbox; <code>mbsync</code> está pensado para sincronizar IMAP↔Maildir y trabajar en modo IMAP desconectado. <sup>28</sup> |
| Parsing de correo       | stdlib de Python                          | <code>extract-msg</code> para Outlook    | <code>email</code> y <code>mailbox</code> son suficientemente potentes para <code>.eml</code> , Maildir y mbox; <code>extract-msg</code> cubre <code>.msg</code> . <sup>29</sup>     |
| Parsing HTML            | <code>BeautifulSoup</code>                | <code>lxml</code>                        | Para plantillas bancarias, <code>BeautifulSoup</code> + selectores CSS suele ser más corto; <code>lxml</code> es excelente si quieres XPath y más control. <sup>19</sup>             |
| PDF nativo              | <code>pypdf</code> + <code>PyMuPDF</code> | <code>pdfplumber</code>                  | <code>pypdf</code> insiste en no sustituir extracción nativa por OCR cuando el PDF ya trae texto. PyMuPDF y pdfplumber ayudan con orden, bloques y tablas. <sup>30</sup>             |
| OCR de PDF              | <code>OCRmyPDF</code>                     | Tesseract puro                           | OCRmyPDF añade la capa OCR al PDF y documenta instalación en Linux/macOS y también en Windows nativo, WSL o Docker. <sup>31</sup>                                                    |
| OCR de imagen           | Tesseract + <code>pytesseract</code>      | NAPS2 para flujo GUI                     | Tesseract es el motor base; NAPS2 es muy cómodo cuando el documento entra por escáner y quieres OCR local en interfaz gráfica. <sup>32</sup>                                         |
| Preprocesado de imagen  | <code>Pillow</code> + <code>OpenCV</code> | —                                        | Umbralizado y limpieza antes del OCR mejoran resultados en tickets y capturas. <sup>33</sup>                                                                                         |

| Etapas                      | Opción recomendada    | Alternativas                                    | Nota práctica                                                                                                               |
|-----------------------------|-----------------------|-------------------------------------------------|-----------------------------------------------------------------------------------------------------------------------------|
| Base canónica local         | SQLite                | SQLCipher si quieres cifrado en BD              | SQLite no requiere servidor separado y es ideal para staging offline. <sup>34</sup>                                         |
| Inspección/depuración de BD | DB Browser for SQLite | SQLiteStudio                                    | Ambas son locales y cómodas para inspeccionar tablas, índices, datos y SQL. <sup>35</sup>                                   |
| Match difuso / dedupe       | RapidFuzz             | <code>difflib</code> si quieres mínimo absoluto | RapidFuzz ofrece matching difuso rápido y suficientemente serio para deduplicar merchants y textos parecidos. <sup>36</sup> |

Si tu requisito de “sin internet” es **estrictamente air-gapped**, entonces el mismo diseño sigue funcionando, pero el correo no se obtiene por IMAP/POP3 en la máquina financiera; se lleva por exportación `.mbox/ .eml` desde otra máquina o por copia física a la carpeta de ingestión. Si el requisito significa “sin nube y sin procesamiento remoto”, entonces Thunderbird/getmail6/mbsync encajan perfectamente: sincronizas, cortas red y procesas todo localmente. <sup>37</sup>

## Modelo canónico, deduplicación y conciliación

### Esquema canónico recomendado

La base canónica no debería intentar copiar MMEX tal cual. Debería ser más rica y más cruda, para no perder información. Mi propuesta mínima es esta:

- `tx_uid`: identificador estable de la transacción en tu pipeline.
- `owner`: `yo`, `esposa`, `joint`.
- `source_type`: `email`, `csv`, `ofx`, `qif`, `mt940`, `pdf`, `image`, `manual`.
- `source_file`, `message_id`, `attachment_sha256`, `content_sha256`.
- `event_date`, `booking_date`, `posted_date` si aplica.
- `amount`, `currency`, `direction` (`debit` / `credit`).
- `account_alias`, `account_number_masked`, `card_last4`.
- `merchant_raw`, `merchant_norm`.
- `tx_type`: `purchase`, `bill_payment`, `external_transfer_in`, `external_transfer_out`, `internal_transfer`, `refund`, `cash_withdrawal`, `fee`.
- `reference_raw`, `reference_norm`.
- `memo`, `raw_text`, `ocr_confidence`, `parser_name`, `parser_version`.
- `category_guess`, `subcategory_guess`, `tags_json`.
- `reconcile_status`, `mmex_account_id`, `mmex_tx_id`.
- `created_at`, `updated_at`.

Ese diseño te da tres cosas que MMEX, por sí solo, no te resuelve igual de bien: **idempotencia**, **fusión multi-fuente** y **auditoría de origen**. Además, MMEX sí tiene sitio de llegada para bastante de esto — transacciones, cuentas, payees, tags, adjuntos y custom fields—, pero no conviene que tu staging dependa del esquema interno exacto de una sola app. <sup>5</sup>

Un esquema inicial en SQLite puede ser tan simple como esto:

```

CREATE TABLE canonical_tx (
  tx_uid TEXT PRIMARY KEY,
  owner TEXT NOT NULL,           -- yo | esposa | joint
  source_type TEXT NOT NULL,
  -- email | csv | ofx | qif | mt940 | pdf | image | manual
  source_file TEXT,
  message_id TEXT,
  attachment_sha256 TEXT,
  content_sha256 TEXT,
  event_date TEXT,
  booking_date TEXT,
  posted_date TEXT,
  amount REAL NOT NULL,
  currency TEXT NOT NULL,
  direction TEXT NOT NULL,      -- debit | credit
  account_alias TEXT,
  account_number_masked TEXT,
  card_last4 TEXT,
  merchant_raw TEXT,
  merchant_norm TEXT,
  tx_type TEXT NOT NULL,
  reference_raw TEXT,
  reference_norm TEXT,
  memo TEXT,
  raw_text TEXT,
  ocr_confidence REAL,
  parser_name TEXT,
  parser_version TEXT,
  category_guess TEXT,
  subcategory_guess TEXT,
  tags_json TEXT,
  reconcile_status TEXT DEFAULT 'new',
  mmex_account_id INTEGER,
  mmex_tx_id INTEGER,
  created_at TEXT DEFAULT CURRENT_TIMESTAMP,
  updated_at TEXT DEFAULT CURRENT_TIMESTAMP
);

CREATE INDEX idx_canonical_tx_hash ON canonical_tx(content_sha256);
CREATE INDEX idx_canonical_tx_match ON canonical_tx(
  owner, amount, currency, merchant_norm, posted_date, booking_date,
  reference_norm
);

```

## Reglas de deduplicación

La deduplicación tiene que ser **determinista primero** y **difusa después**. La secuencia que recomiendo es:

1. **Hash exacto** del contenido crudo o del adjunto. Si coincide, es el mismo artefacto.

2. **Fingerprint fuerte** de negocio:

`owner + amount + currency + normalized_reference + normalized_account/card + date_bucket`.

3. **Fingerprint medio**: `owner + amount + currency + merchant_norm + ±1-3 días`.

4. **Matching difuso** del merchant y memo con RapidFuzz solo cuando los pasos previos no resuelven. <sup>36</sup>

Las reglas de merge deben ser conservadoras. Mi propuesta:

- **Banco vence a correo** en `booking_date`, `posted_date`, monto final y referencia formal.
- **Correo vence a banco** en texto descriptivo legible del comercio, asunto del pago y evidencia humana.
- **Imagen/OCR** solo rellena huecos; no debería derrotar a OFX/MT940/CSV salvo que no exista otra fuente.
- Si dos fuentes discrepan en monto o dirección, no se fusionan: se marcan para revisión.
- La entidad `owner` no debe perderse nunca: una compra idéntica por monto y fecha de ambos no es duplicado si cae en tarjetas distintas.

## Reglas de conciliación para tu caso de uso

Aquí está el corazón práctico:

- **Compra con tarjeta**: si llega email y también aparece en exporte bancario, fusiona en una sola transacción canónica. Conserva el texto completo del correo en `raw_text`.
- **Pago de cuenta o servicio**: si el pago sale a un tercero no modelado como cuenta MMEX, guárdalo como gasto/withdrawal categorizado.
- **Transferencia externa**: si el otro extremo no existe como cuenta en tu archivo MMEX, representa el movimiento como retiro o depósito, no como transferencia interna.
- **Transferencia interna**: si ambos extremos son cuentas que existen en tu base MMEX —por ejemplo corriente ↔ tarjeta, o cuenta tuya ↔ cuenta de tu esposa si ambas están modeladas— entonces usa **transferencia interna** con `TOACCOUNTID`.
- **Pago de tarjeta**: si mantienes la tarjeta como cuenta MMEX separada, el pago de la tarjeta debería entrar como **transferencia interna**, no como gasto, para no duplicar el consumo ya registrado por las compras.
- **Reembolsos y reversiones**: jamás sobrescribas la compra original; crea una transacción separada de abono asociable por referencia o tag.
- **Correos de autorización vs liquidación**: si el correo describe una preautorización y el banco liquida dos días después con descriptor distinto, fusiona solo si coinciden monto, sentido, tarjeta y una similitud razonable del merchant; si no, deja *followup*.

MMEX ya contempla estados como *Unreconciled*, *Reconciled*, *Void*, *Followup* y *Duplicate*, lo que te permite marcar salidas del pipeline para revisión humana sin borrar trazabilidad. <sup>38</sup>

## Actualización de .mmb y capa de conversión

### Veredicto técnico

La ruta **más segura** para la primera implementación es:

1. Ingerir y normalizar todo en SQLite local.
2. Exportar a **CSV compatible con MMEX** por cada cuenta de destino.



3. Importar en MMEX y revisar *Followup/Duplicate*.
4. Solo cuando el flujo ya esté estable, habilitar **writer directo a** `.mmb`.

La razón es simple: la documentación oficial de MMEX lista importación/exportación en CSV y QIF, y el importador CSV admite orden flexible de campos. Además, la propia documentación explica que tras importar, las transacciones quedan en estado *Followup*. Es una “zona de aterrizaje blanda” excelente para una fase 1. <sup>39</sup>

### Tabla de mapeo sugerida

| Campo banco / origen                           | Campo canónico                                               | Destino MMEX                                                              | Nota                                                                                                                                                                |
|------------------------------------------------|--------------------------------------------------------------|---------------------------------------------------------------------------|---------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Fecha operación / fecha valor / booking date   | <code>posted_date</code> / <code>booking_date</code>         | <code>CHECKINGACCOUNT_V1.TRANSDATE</code>                                 | Si tienes dos fechas, usa una como fecha contable y conserva la otra en custom field <code>source_booking_date</code> . <sup>40</sup>                               |
| Importe                                        | <code>amount</code> + <code>direction</code>                 | <code>CHECKINGACCOUNT_V1.TRANSAMOUNT</code> + <code>TRANSCODE</code>      | En MMEX conviene validar el patrón exacto de tu versión creando una transacción manual y replicando su combinación de signo + <code>TRANSCODE</code> . <sup>5</sup> |
| Moneda                                         | <code>currency</code>                                        | Cuenta ( <code>ACCOUNTLIST_V1.CURRENCYID</code> ) + custom field          | MMEX trabaja a nivel de cuenta; si el origen trae moneda original distinta, guárdala como campo personalizado. <sup>5</sup>                                         |
| Merchant / payee / descripción                 | <code>merchant_norm</code>                                   | <code>PAYEE_V1.PAYEENAME</code> + <code>CHECKINGACCOUNT_V1.PAYEEID</code> | Conviene crear/normalizar payees antes de insertar. <sup>5</sup>                                                                                                    |
| Categoría del banco / MCC                      | <code>category_guess</code> / <code>subcategory_guess</code> | <code>CATEGID</code> / <code>SUBCATEGID</code>                            | Puede quedar vacío en la primera importación y completarse luego. <sup>5</sup>                                                                                      |
| N.º de operación / referencia / end-to-end ref | <code>reference_norm</code>                                  | <code>TRANSACTIONNUMBER</code> o custom field <code>source_ref</code>     | Para MT940, el campo <code>86</code> puede traer referencias estructuradas útiles. <sup>40</sup>                                                                    |

| Campo banco / origen                                                                                                                                 | Campo canónico             | Destino MMEX                                      | Nota                                                                                                            |
|------------------------------------------------------------------------------------------------------------------------------------------------------|----------------------------|---------------------------------------------------|-----------------------------------------------------------------------------------------------------------------|
| Cuenta origen/ destino / IBAN / BBAN                                                                                                                 | <code>account_alias</code> | <code>ACCOUNTID</code> / <code>TOACCOUNTID</code> | <code>TOACCOUNTID</code> solo si de verdad es transferencia interna entre cuentas MMEX. <sup>40</sup>           |
| Tarjeta últimos 4                                                                                                                                    | <code>card_last4</code>    | custom field o tag                                | No es un campo transaccional nativo obvio en <code>CHECKINGACCOUNT_V1</code> ; mejor custom field. <sup>5</sup> |
| Texto original del correo/ PDF/OCR                                                                                                                   | <code>raw_text</code>      | custom field, <code>NOTES</code> y/o adjunto      | MMEX soporta adjuntos y campos personalizados; mejor no meter todo en una sola nota larga. <sup>41</sup>        |
| Etiquetas<br><code>yo</code> ,<br><code>esposa</code> ,<br><code>joint</code> ,<br><code>correo</code> ,<br><code>ocr</code> ,<br><code>banco</code> | <code>tags_json</code>     | <code>TAG_V1</code> + <code>TAGLINK_V1</code>     | Muy útil para separar origen y responsable sin romper categorías. <sup>42</sup>                                 |

## Ejemplos de parsing local

### Correo local (`.eml`, Maildir, mbox) y extracción de adjuntos

```

from email import policy
from email.parser import BytesParser
import mailbox
import hashlib
from pathlib import Path

def sha256_bytes(data: bytes) -> str:
    return hashlib.sha256(data).hexdigest()

def extract_message_parts(msg):
    text_parts = []
    attachments = []

    for part in msg.walk():
        ctype = part.get_content_type()
        filename = part.get_filename()
        payload = part.get_payload(decode=True) or b""

```

```

    if filename:
        attachments.append({
            "filename": filename,
            "sha256": sha256_bytes(payload),
            "bytes": payload,
            "content_type": ctype,
        })
    elif ctype == "text/plain":
        text_parts.append(payload.decode(part.get_content_charset() or
"utf-8", errors="replace"))
    elif ctype == "text/html":
        text_parts.append(payload.decode(part.get_content_charset() or
"utf-8", errors="replace"))

    return "\n".join(text_parts), attachments

def iter_maildir(maildir_path: str):
    md = mailbox.Maildir(maildir_path, factory=None)
    for key, raw in md.iteritems():
        if isinstance(raw, bytes):
            msg = BytesParser(policy=policy.default).parsebytes(raw)
        else:
            # mailbox may already give a Message object
            msg = raw
        text, attachments = extract_message_parts(msg)
        yield {
            "message_id": msg.get("Message-ID"),
            "date": msg.get("Date"),
            "from": msg.get("From"),
            "subject": msg.get("Subject"),
            "raw_text": text,
            "attachments": attachments,
        }

```

La base documental oficial para esto está en `mailbox` y `email` de Python; si necesitas conectarte por protocolo, `imaplib` y `poplib` también tienen documentación oficial en español. 18

### CSV, OFX y MT940 hacia el esquema canónico

```

import csv
from decimal import Decimal
from pathlib import Path

def parse_csv_bank(path):
    with open(path, "r", encoding="utf-8-sig", newline="") as f:
        reader = csv.DictReader(f)
        for row in reader:
            yield {
                "source_type": "csv",
                "posted_date": row.get("Fecha") or row.get("Date"),
            }

```

```

        "amount": Decimal((row.get("Monto") or row.get("Amount") or
"0").replace(".", "").replace(",", ".")),
        "currency": row.get("Moneda") or row.get("Currency") or
"CLP",
        "merchant_raw": row.get("Descripción") or
row.get("Description"),
        "reference_raw": row.get("Referencia") or
row.get("Reference"),
    }

def parse_ofx(path):
    from ofxtools import OFXTree
    tree = OFXTree()
    tree.parse(path)
    ofx = tree.convert()
    for stmt in getattr(ofx, "bankmsgsrsrv1", []).stmttrnrs:
        for tx in stmt.stmtrs.banktrnlist.stmttrn:
            yield {
                "source_type": "ofx",
                "posted_date": tx.dtposted.strftime("%Y-%m-%d") if
tx.dtposted else None,
                "amount": tx.trnamt,
                "currency": getattr(stmt.stmtrs.curdef, "value", None) or
"CLP",
                "merchant_raw": tx.name or tx.memo,
                "reference_raw": tx.fitid,
            }

def parse_mt940_file(path):
    import mt940
    data = Path(path).read_text(encoding="utf-8", errors="replace")
    transactions = mt940.parse(data)
    for tx in transactions:
        yield {
            "source_type": "mt940",
            "posted_date": str(tx.data.get("date") or
tx.data.get("entry_date") or ""),
            "amount": tx.amount.amount,
            "currency": tx.amount.currency,
            "merchant_raw": tx.data.get("transaction_details"),
            "reference_raw": tx.data.get("transaction_reference"),
        }

```

OFX está pensado precisamente para intercambio de datos financieros estructurados; `ofxtools` y `libofx` son opciones maduras. En MT940, los campos `61` y `86` son especialmente valiosos para fecha/monto/dirección y texto adicional estructurado. <sup>43</sup>

**OCR local para imagen o PDF escaneado**

```

from PIL import Image
import pytesseract

def ocr_image(path, lang="spa+eng"):
    img = Image.open(path)
    text = pytesseract.image_to_string(img, lang=lang)
    return text

```

Tesseract funciona localmente con paquetes de idioma instalados; para PDF escaneado, OCRmyPDF suele ser mejor porque deja el PDF buscable y conserva la capa original. <sup>44</sup>

## Writer directo a `.mmb`

El writer directo conviene hacerlo **solo** después de crear, en tu archivo real, una transacción manual de cada tipo y estudiar qué guarda tu versión de MMEX en `CHECKINGACCOUNT_V1`. Aun así, la estructura general es esta:

```

import sqlite3
from datetime import datetime

def get_or_create_payee(conn, payee_name: str) -> int:
    cur = conn.execute("SELECT PAYEEID FROM PAYEE_V1 WHERE PAYEENAME = ?",
    (payee_name,))
    row = cur.fetchone()
    if row:
        return row[0]
    cur = conn.execute(
        "INSERT INTO PAYEE_V1 (PAYEENAME, CATEGID, SUBCATEGID) VALUES (?, -1, -1)",
        (payee_name, )
    )
    return cur.lastrowid

def insert_mmex_tx(conn, tx, account_id: int, category_id: int = -1,
    subcategory_id: int = -1,
    to_account_id: int = -1):
    payee_id = get_or_create_payee(conn, tx["merchant_norm"] or
    tx["merchant_raw"] or "Desconocido")

    # Ajusta el patrón de monto/signo según un registro semilla creado
    # manualmente en tu MMEX.
    transcode = {
        "purchase": "Withdrawal",
        "bill_payment": "Withdrawal",
        "external_transfer_out": "Withdrawal",
        "external_transfer_in": "Deposit",
        "refund": "Deposit",
        "internal_transfer": "Transfer",
    }[tx["tx_type"]]

```

```

cur = conn.execute("""
    INSERT INTO CHECKINGACCOUNT_V1 (
        ACCOUNTID, TOACCOUNTID, PAYEEID, TRANSCODE, TRANSAMOUNT,
        STATUS, TRANSACTIONNUMBER, NOTES, CATEGID, SUBCATEGID,
        TRANSDATE, LASTUPDATEDTIME, FOLLOWUPID, TOTRANSAMOUNT
    ) VALUES (?, ?, ?, ?, ?, ?, ?, ?, ?, ?, ?, ?, ?, ?)
""", (
    account_id,
    to_account_id,
    payee_id,
    transcode,
    float(tx["amount"]),
    "Followup",
    tx.get("reference_norm") or tx.get("reference_raw"),
    tx.get("memo"),
    category_id,
    subcategory_id,
    tx["posted_date"],
    datetime.utcnow().strftime("%Y-%m-%dT%H:%M:%S"),
    0,
    float(tx["amount"]),
))
return cur.lastrowid

def safe_update_mmex(mmex_path, txs):
    conn = sqlite3.connect(mmex_path)
    try:
        conn.execute("BEGIN IMMEDIATE")
        # Validación mínima del esquema
        tables = {r[0] for r in conn.execute(
            "SELECT name FROM sqlite_master WHERE type='table'"
        )}
        required = {"CHECKINGACCOUNT_V1", "PAYEE_V1", "ACCOUNTLIST_V1"}
        if not required.issubset(tables):
            raise RuntimeError("El archivo no parece una base MMEX compatible.")

        for tx in txs:
            insert_mmex_tx(conn, tx, account_id=tx["mmex_account_id"])
        conn.commit()
    except Exception:
        conn.rollback()
        raise
    finally:
        conn.close()

```

Esta es la observación clave: el esquema público de MMEX permite esta ruta, pero la *semántica exacta* de ciertos campos —especialmente transferencias internas, signos, `TOTRANSAMOUNT`, estados y algunos defaults— conviene replicarla desde registros “semilla” creados por la propia UI de tu versión. Eso baja el riesgo de incompatibilidades silenciosas. 5

## Capa de conversión si prefieres no escribir directo

Si quieres una fase 1 muy conservadora, exporta a un CSV orientado al importador flexible de MMEX:

```
import csv

def export_mmex_import_csv(path, rows):
    with open(path, "w", encoding="utf-8-sig", newline="") as f:
        w = csv.writer(f)
        w.writerow(["Date", "Payee", "Amount (+/-)", "Category",
                    "SubCategory", "Remarks", "Number"])
        for r in rows:
            signed_amount = r["amount"] if r["direction"] == "credit" else -
r["amount"]
            w.writerow([
                r["posted_date"],
                r["merchant_norm"] or r["merchant_raw"],
                signed_amount,
                r.get("category_guess") or "",
                r.get("subcategory_guess") or "",
                r.get("memo") or "",
                r.get("reference_norm") or r.get("reference_raw") or "",
            ])
```

MMEX documenta que el importador CSV acepta orden libre de campos y que puede trabajar con `Date`, `Payee`, `Amount (+/-)`, `Category`, `Subcategory`, `Remarks`, `Number`, `Debit` y `Credit`; además, ese flujo es especialmente cómodo cuando quieres importar por cuenta y dejar todo marcado como *Followup* para revisión final. <sup>45</sup>

## Seguridad, implementación y pruebas

### Seguridad y privacidad

La estrategia de seguridad más práctica para tu escenario no es “complejidad máxima”, sino **capas simples y locales**:

- **Cifra el contenedor de trabajo**, no solo el archivo final. Si tu staging y tus adjuntos quedan en texto claro fuera de la base, cifrar solo `.mmb` no basta.
- Para cifrado local multiplataforma, **VeraCrypt** sigue siendo una opción sólida de contenedor o volumen cifrado; si quieres cifrar la base SQLite de staging como base de datos, **SQLCipher** es la extensión clásica. <sup>46</sup>
- Si usas MMEX con `.mmb` plano, una opción muy sensata es guardar **todo el workspace** — staging DB, `.mmb`, adjuntos, backups— dentro de un volumen cifrado. Si prefieres seguridad nativa de MMEX, al final puedes convertir a `.emb`, pero yo no basaría la automatización diaria en escribir directamente el `.emb`. <sup>10</sup>
- Para copias de seguridad locales cifradas, **restic** encaja muy bien: es local, sin servidor, y cifra el repositorio. MMEX además recomienda hacer respaldos regulares y mantener copias separadas.

<sup>47</sup>

- Usa **cuentas de sistema separadas** o un perfil dedicado al trabajo financiero. Y guarda los adjuntos dentro de la carpeta de adjuntos de MMEX o en una carpeta hermana cifrada, nunca dispersos en descargas. MMEX documenta que los adjuntos se copian a una carpeta definida por el usuario y no quedan como meros enlaces. <sup>48</sup>

## Plan de implementación paso a paso

| Fase          | Tarea                                                                                                                                            | Esfuerzo estimado |
|---------------|--------------------------------------------------------------------------------------------------------------------------------------------------|-------------------|
| Preparación   | Instalar Python, SQLite tools, MMEX, Tesseract, PDF tools, y elegir modo de correo (Thunderbird o fetcher headless)                              | 0.5–1 día         |
| Ingestión     | Sincronizar/exportar correo a Maildir/mbox y crear carpetas <code>inbox/raw</code> , <code>inbox/bank_exports</code> , <code>inbox/manual</code> | 0.5 día           |
| Adaptadores   | Implementar adaptadores para 1–2 bancos prioritarios: correo + CSV/OFX/MT940 más usado                                                           | 2–4 días          |
| Staging       | Crear base SQLite canónica, hashes, reglas de merge y tagging <code>yo/esposa/joint</code>                                                       | 1–2 días          |
| Conciliación  | Definir reglas de compras, pagos de cuenta, transferencias externas e internas                                                                   | 1–2 días          |
| Salida a MMEX | Empezar con exportador CSV por cuenta; dejar writer directo a <code>.mmb</code> para segunda iteración                                           | 1–2 días          |
| Hardening     | Backups, cifrado, pruebas de restauración, checklist de idempotencia                                                                             | 1 día             |

En total, un **MVP serio** te quedarías razonablemente entre **1 y 2 semanas de trabajo parcial, o 4–7 días efectivos** si ya te mueves cómodo con Python/SQLite. La parte que más tiempo suele pedir no es OFX ni MT940; son las plantillas de correos y las excepciones de banco. Ese es el sitio donde el humor se vuelve contable: el banco te jura que exporta “CSV”, pero en realidad exporta “sorpresa delimitada por ansiedad”.

## Checklist de pruebas

Antes de confiarle tu `.mmb` real al pipeline, yo validaría esto:

1. **Idempotencia:** volver a correr el import no debe duplicar nada.
2. **Cobertura mínima de fuentes:** al menos un caso por correo, CSV, OFX/QIF/MT940, PDF y manual.
3. **Dos usuarios:** compra de cada uno, una compra en cuenta conjunta, y una transferencia entre ambos.
4. **Transferencias:** una interna MMEX↔MMEX y una externa a tercero.
5. **Pagos de tarjeta:** que no dupliquen gasto si ya registraste compras en la cuenta de tarjeta.
6. **Fechas:** discrepancia entre fecha de autorización y fecha contable.
7. **Monedas:** al menos una transacción con moneda distinta de la cuenta.
8. **OCR:** ticket legible, ticket mediocre y captura de pantalla.
9. **Rollback:** restaurar backup funcional del `.mmb` y del staging.
10. **Adjuntos:** verificar que el texto original o el archivo fuente quede trazable.



## Límites y preguntas abiertas

Hay dos límites que conviene dejar explícitos. El primero: **todo este informe asume que tu `.mmb` es de MMEX**. La documentación oficial de MMEX es suficiente para sostener esa ruta, pero si tu archivo `.mmb` viniera de otro software, cambiaría por completo el procedimiento de escritura directa. <sup>49</sup>

El segundo: aunque MMEX documenta `.mmb` y `.emb`, la parte verdaderamente delicada no es leer el esquema, sino **replicar fielmente los valores que tu versión concreta de MMEX espera en inserciones directas**. Por eso, mi recomendación final sigue siendo esta: **fase 1 con staging SQLite + import CSV a MMEX; fase 2, solo si ya estás cómodo, writer directo a `.mmb`**. En ingeniería financiera local, eso no es ser conservador: es ahorrar una sesión de arqueología digital a las dos de la mañana.

---

<sup>1</sup> <sup>4</sup> <sup>9</sup> <sup>25</sup> <sup>49</sup> Manual de usuario de Money Manager Ex

[https://moneymanagerex.org/moneymanagerex/index.html?lang=es\\_ES&utm\\_source=chatgpt.com](https://moneymanagerex.org/moneymanagerex/index.html?lang=es_ES&utm_source=chatgpt.com)

<sup>2</sup> <sup>12</sup> <sup>15</sup> <sup>43</sup> OFX Work Group - Financial Data Exchange

<https://financialdataexchange.org/about-fdx/ofx-work-group/>

<sup>3</sup> <sup>27</sup> <sup>37</sup> IMAP Synchronization | Thunderbird Help

<https://support.mozilla.org/en-US/kb/imap-synchronization>

<sup>5</sup> <sup>6</sup> <sup>7</sup> database/tables.sql at master · moneymanagerex/database · GitHub

<https://github.com/moneymanagerex/database/blob/master/tables.sql>

<sup>8</sup> <sup>11</sup> <https://kaitai.io/>

<https://kaitai.io/>

<sup>10</sup> <sup>42</sup> <https://moneymanagerex.org/moneymanagerex/>

<https://moneymanagerex.org/moneymanagerex/>

<sup>13</sup> <https://www.rfc-editor.org/rfc/rfc4180.html>

<https://www.rfc-editor.org/rfc/rfc4180.html>

<sup>14</sup> <sup>18</sup> <sup>24</sup> <sup>29</sup> <https://docs.python.org/es/3/library/mailbox.html>

<https://docs.python.org/es/3/library/mailbox.html>

<sup>16</sup> <https://info.quicken.com/win/why-can-t-i-import-data-from-a-qif-file-into-a-ban>

<https://info.quicken.com/win/why-can-t-i-import-data-from-a-qif-file-into-a-ban>

<sup>17</sup> <sup>40</sup> [https://danskeci.com/-/media/pdf/danskeci-com/swift-mt/reconciliation/mt940\\_structured.pdf?](https://danskeci.com/-/media/pdf/danskeci-com/swift-mt/reconciliation/mt940_structured.pdf?hash=C4A798C2B633C25AE37B5BCBA950605B&rev=eaffaec517704cea9e0689cde0fd626c)

[hash=C4A798C2B633C25AE37B5BCBA950605B&rev=eaffaec517704cea9e0689cde0fd626c](https://danskeci.com/-/media/pdf/danskeci-com/swift-mt/reconciliation/mt940_structured.pdf?hash=C4A798C2B633C25AE37B5BCBA950605B&rev=eaffaec517704cea9e0689cde0fd626c)

[https://danskeci.com/-/media/pdf/danskeci-com/swift-mt/reconciliation/mt940\\_structured.pdf?](https://danskeci.com/-/media/pdf/danskeci-com/swift-mt/reconciliation/mt940_structured.pdf?hash=C4A798C2B633C25AE37B5BCBA950605B&rev=eaffaec517704cea9e0689cde0fd626c)

[hash=C4A798C2B633C25AE37B5BCBA950605B&rev=eaffaec517704cea9e0689cde0fd626c](https://danskeci.com/-/media/pdf/danskeci-com/swift-mt/reconciliation/mt940_structured.pdf?hash=C4A798C2B633C25AE37B5BCBA950605B&rev=eaffaec517704cea9e0689cde0fd626c)

<sup>19</sup> <https://www.crummy.com/software/BeautifulSoup/bs4/doc/>

<https://www.crummy.com/software/BeautifulSoup/bs4/doc/>

<sup>20</sup> [https://msg-extractor.readthedocs.io/en/stable/extract\\_msg.html](https://msg-extractor.readthedocs.io/en/stable/extract_msg.html)

[https://msg-extractor.readthedocs.io/en/stable/extract\\_msg.html](https://msg-extractor.readthedocs.io/en/stable/extract_msg.html)

<sup>21</sup> <sup>30</sup> <https://pypdf.readthedocs.io/en/stable/user/extract-text.html>

<https://pypdf.readthedocs.io/en/stable/user/extract-text.html>

<sup>22</sup> <sup>32</sup> <sup>44</sup> <https://tesseract-ocr.github.io/tessdoc/Installation.html>

<https://tesseract-ocr.github.io/tessdoc/Installation.html>

- 23 26 39 <https://moneymanagerex.org/docs/features/importexport/>  
<https://moneymanagerex.org/docs/features/importexport/>
- 28 <https://getmail6.org/documentation.html>  
<https://getmail6.org/documentation.html>
- 31 <https://ocrmypdf.readthedocs.io/>  
<https://ocrmypdf.readthedocs.io/>
- 33 <https://pillow.readthedocs.io/en/stable/reference/Image.html>  
<https://pillow.readthedocs.io/en/stable/reference/Image.html>
- 34 <https://docs.python.org/es/3/library/sqlite3.html>  
<https://docs.python.org/es/3/library/sqlite3.html>
- 35 <https://sqlitebrowser.org/>  
<https://sqlitebrowser.org/>
- 36 RapidFuzz 3.14.5 documentation  
[https://rapidfuzz.github.io/RapidFuzz/?utm\\_source=chatgpt.com](https://rapidfuzz.github.io/RapidFuzz/?utm_source=chatgpt.com)
- 38 <https://moneymanagerex.org/docs/features/transactions/>  
<https://moneymanagerex.org/docs/features/transactions/>
- 41 48 <https://moneymanagerex.org/docs/features/attachments/>  
<https://moneymanagerex.org/docs/features/attachments/>
- 45 Manuel de l'utilisateur de Money Manager Ex  
[https://moneymanagerex.org/moneymanagerex/fr\\_FR/index.html?utm\\_source=chatgpt.com](https://moneymanagerex.org/moneymanagerex/fr_FR/index.html?utm_source=chatgpt.com)
- 46 <https://veracrypt.jp/>  
<https://veracrypt.jp/>
- 47 <https://restic.readthedocs.io/>  
<https://restic.readthedocs.io/>